# ECO SURVIVOR

Jeu d'arcade 2D en vue du dessus réalisé avec **Godot 4** et **GDScript**. Projet de Terminale STI2D, spécialité SIN.

**Jouer en ligne :** https://mou.alwaysdata.net/jeu-dd/

## Auteurs


- Rayan MOUAK
- Classe : Terminale STI2D SIN Année scolaire : 2026-2027

Projet individuel.


## Thème de développement durable

La ville est envahie par les déchets et la pollution. Le joueur incarne un agent écologique équipé d'une pince à déchets. Il doit **ramasser les déchets recyclables**, **éviter les dangers environnementaux** et **économiser son énergie**.

Le jeu montre plusieurs idées du développement durable :

- le **tri des déchets** : chaque matière a une valeur différente (les déchets électroniques, les plus polluants s'ils ne sont pas recyclés, rapportent le plus) ;
- l'**économie des ressources** : l'énergie du sprint est limitée, il faut l'utiliser au bon moment ;
- les **conséquences de la pollution** : plus le temps passe, plus la ville se pollue (brume, nuages toxiques, circulation plus dense) ;
- la ville du futur : toits solaires, toits végétalisés, éoliennes et jardins partagés, à côté d'usines qui fument.

## Principe du jeu

- **Objectif** : atteindre le score demandé (**1000 points** en mode Normal) **avant la fin du chronomètre** (**2 min 30** en mode Normal).
- Le joueur a **3 vies**. Chaque vie a une **barre de santé** de 100 points. Quand la santé tombe à 0, il perd une vie et réapparaît au centre de la ville.

| Objet | Effet |
|---|---|
| Plastique (bouteille bleue) | +10 points |
| Papier (journal) | +15 points |
| Verre (bouteille verte) | +20 points |
| Déchet électronique (téléphone) | +30 points (disparaît plus vite) |
| Cœur | +35 santé |
| Batterie | énergie rechargée à 100 % |
| Déchet toxique (baril jaune) | **−50 points et −25 santé** |
| Nuage de pollution | perte de santé tant qu'on reste dedans |
| Voiture | −20 santé et recul |
| Bâtiments, arbres, fontaine, étangs, conteneurs | obstacles |

Les objets apparaissent à des endroits aléatoires et disparaissent au bout de quelques secondes (ils clignotent avant). Un nuage de pollution est annoncé par un **cercle rouge clignotant** avant de devenir dangereux.

**Difficulté progressive** : la partie est découpée en **5 niveaux de pollution** (un nouveau niveau toutes les 30 s). À chaque niveau, une alarme retentit : plus de déchets, plus de déchets toxiques et de nuages, des nuages plus grands, plus de voitures et plus rapides, et une brume de pollution de plus en plus forte.

Le menu OPTIONS propose 3 difficultés : Facile (800 pts en 3 min), Normal (1000 pts en 2 min 30), Difficile (1300 pts en 2 min 30).

## Moteur choisi et justification

J'ai choisi **Godot 4** :

- il est **gratuit** et libre (licence MIT), aucun outil payant n'est nécessaire ;
- il est fait pour la **2D** (CharacterBody2D, Area2D, collisions, particules, caméra…) ;
- le langage **GDScript** ressemble à Python ;
- le système de **scènes** et de **signaux** permet un projet bien organisé ;
- il permet d'**exporter le jeu pour le Web** (rendu Compatibility, sans Thread Support).

### Organisation du projet

```
ECO_SURVIVOR/
├── project.godot          configuration (scène de départ, touches, autoloads, rendu
Compatibility)
├── export_presets.cfg     export Web (Thread Support désactivé)
├── scenes/                toutes les scènes (.tscn)
├── scripts/               tous les scripts GDScript (.gd)
├── ui/theme.tres          thème de l'interface (couleurs, polices, boutons)
├── assets/                images, sons, polices, shaders
├── tools/                 programmes Python qui ont créé les images et les sons
└── tests/                 tests automatiques (non inclus dans l'export)
```

| Scène | Nœud racine | Rôle |
|---|---|---|
| `MainMenu.tscn` | Control | menu principal : JOUER, COMMENT JOUER, OPTIONS, QUITTER |
| `Main.tscn` | Node2D | la partie : ville, obstacles, voies des voitures, joueur, HUD, pause |
| `Player.tscn` | CharacterBody2D | le joueur (sprite animé, collision, caméra) |
| `Collectible.tscn` | Area2D | déchet ou bonus à ramasser |
| `Pollution.tscn` | Area2D | nuage de pollution |
| `ToxicWaste.tscn` | Area2D | baril toxique |
| `Car.tscn` | Area2D | voiture qui suit une voie |
| `HUD.tscn` | CanvasLayer | score, objectif, temps, vies, santé, énergie, déchets |
| `PauseMenu.tscn` / `OptionsPanel.tscn` | — | pause et options |
| `Victory.tscn` / `GameOver.tscn` | Control | écrans de fin |

Les scènes communiquent par des **signaux** : un déchet envoie `collected`, un baril `touched`, une voiture `hit_player`, le joueur `died` / `health_changed`… Le script `game_manager.gd` reçoit ces signaux, met à jour le score et les vies, puis prévient le HUD avec ses propres signaux (`score_changed`, `time_changed`, `lives_changed`…).

Fonctions principales :

- `player.gd` → `_physics_process()` (touches, accélération, sprint, `move_and_slide()`), `take_damage()` (santé, recul, invincibilité courte).
- `game_manager.gd` → `find_free_position()` (position libre grâce au moteur physique), les fonctions `_on_..._timer()` (apparitions), `add_score()` (victoire), `_on_player_died()` (vies, défaite), `_end_game()`. Le tableau `LEVELS` contient les réglages des 5 niveaux de difficulté.

## Commandes

| Action | Clavier | Manette |
|---|---|---|
| Se déplacer | **Z Q S D** ou **flèches** | stick gauche |
| Sprinter (consomme l'énergie) | **Maj** ou **Espace** | bouton A |
| Pause | **Échap** ou **P** | Start |
| Menus | souris, ou flèches + Entrée | |

## Victoire / Défaite

- **Victoire** : le score atteint l'objectif avant la fin du temps. L'écran de victoire donne de 1 à 3 étoiles (+1 s'il reste au moins 2 vies, +1 s'il reste au moins 30 s).
- **Défaite** : le temps est écoulé, **ou** le joueur a perdu ses 3 vies.
- Les deux écrans affichent les statistiques (score, déchets par type, temps, niveau de pollution, record) et les boutons **RECOMMENCER** et **RETOUR AU MENU**.

## Répartition du travail

Projet individuel : je suis seul sur ce projet, il n'y a donc pas de répartition entre plusieurs élèves. Le code et les ressources ont été réalisés avec l'aide d'une IA (voir les sources).

## Bug ou difficulté principale

Pendant la création du jeu, plusieurs problèmes simples sont apparus :

1. **Des déchets apparaissaient dans les bâtiments** : ils étaient placés au hasard sur la carte, parfois à l'intérieur d'une maison, donc impossibles à ramasser.
2. **Le personnage était trop petit** à l'écran : on le voyait mal au milieu de la ville.
3. **Le logo du menu était coupé** : le mot « SURVIVOR » dépassait de l'image.
4. **Une erreur apparaissait sur l'écran Game Over** : le script cherchait les étoiles, qui n'existent que sur l'écran de victoire.

## Solution trouvée

1. La fonction `find_free_position()` vérifie avec le moteur physique (`intersect_shape()`) qu'il n'y a pas d'obstacle à l'endroit choisi. Sinon, elle essaie une autre position.
2. Le sprite du joueur a été agrandi (échelle 0,62 → 0,75) et sa zone de collision aussi.
3. Le logo a été refait en mesurant la largeur du texte avant de le dessiner.
4. Le script utilise `get_node_or_null("%Stars")` : si les étoiles n'existent pas (écran Game Over), il ne fait rien au lieu de provoquer une erreur.

## Amélioration après le test croisé

*(À compléter après avoir fait jouer quelqu'un au jeu.)*

- Testeur : .....
- Ce qu'il a remarqué : .....
- Ce que j'ai changé (ou ce que je changerais) : .....

## Sources des médias et ressources

- **Images** (personnage, déchets, voitures, arbres, ville, icônes, logo) : créées pour ce projet, dessinées par programme avec Python et Pillow (`tools/generate_art.py`, `tools/generate_city.py`).
- **Sons et musiques** : créés pour ce projet, synthétisés avec Python, numpy et scipy (`tools/generate_sounds.py`). Aucun son pris sur Internet.
- **Polices** : *Orbitron* et *Exo 2* (Google Fonts), licence libre SIL Open Font License 1.1 (licences dans `assets/fonts/`).
- **Moteur** : Godot Engine 4, gratuit, licence MIT — https://godotengine.org
- **Aide** : le code, les images et les sons ont été réalisés avec l'aide de l'IA Claude (Anthropic).

## Lancer et exporter le projet

1. Installer **Godot 4** (version 4.3 ou plus récente, version standard).

2. Dans Godot : **Importer** → choisir `project.godot` → **Importer et modifier**.
3. Appuyer sur **F5**.

Export Web : *Projet → Exporter…*, préréglage **Web** déjà configuré (Thread Support désactivé).